

Compte Rendu du Projet

Application Web

Sara Bencheqroune 21441027

Jeremy Zhou 21126962

Groupe 3

1. Introduction :

Ce projet consiste à développer une application web interactive, structurée en deux parties principales : un client réalisé avec ReactJS et un serveur backend utilisant Express.js et MongoDB. L'objectif principal est de permettre aux utilisateurs de créer, gérer et visualiser des projets, tout en facilitant la communication via une session de discussion. Le projet intègre d'abord les fonctionnalités essentielles telles que l'authentification et la déconnexion, ainsi que des fonctionnalités complémentaires comme la gestion des messages et le téléversement d'images.

2. Architecture du projet :

Partie Client (ReactJS)

- composée de plusieurs composants tels que :
 - MainPage, Login, Signin, Profil représentant les pages principales à afficher.
 - Header, Logo, Search, GestionConnexion représentant le header des pages principales.
 - Body, Aside, New_message, List_messages représentant le body de la page MainPage.
 - Ami, Message représentant respectivement un ami et un message.
 - PhotoProfil représentant la photo de profil de l'utilisateur.
 - BanniereProfil représentant la bannière du profil affichant la photo de profil, le nom, le prénom et la biographie de l'utilisateur.
 - BodyProfil représentant le body de la page Profil.
 - BodyInfos, BodyFavoris, BodyListeAmis, BodyListeMsgEnvoyes représentant respectivement le body affichant les informations supplémentaires de l'utilisateur, la liste des messages mis en favoris par l'utilisateur, la liste des amis de l'utilisateur et la liste des messages envoyés par l'utilisateur.
 - LoginForm, SignupForm pour l'authentification.
 - TextAreaField, ImageUpload, ImagePreview pour la gestion du formulaire des projets.
 - pattern et Loader pour améliorer l'expérience visuellement durant les chargements.
 - ProjectList pour afficher la liste des projets disponibles.
- Utilisation de React Router pour gérer la navigation entre les différentes vues (MainContentWrapped, ChatScreen, ChatGeneral).
-

Partie Serveur (Express.js, MongoDB)

- Structure organisée en répertoires :
 - /routes : gestion des différentes routes (utilisateurs, projets, messages).
 - /models : modèles Mongoose définissant le schéma logique des données.

- Connexion sécurisée à MongoDB Atlas via `db.js`.
- Gestion des sessions utilisateurs et communication via API REST (requêtes HTTP)

3. Fonctionnalités réalisées :

Côté Client (dossier src):

- Authentification utilisateur complète (inscription, connexion).
- Formulaire interactif de création de projets avec upload et aperçu d'images.
- Bouton de déconnexion accessible depuis toutes les pages.
- Page dédiée à la création d'un nouveau projet.
- Page dédiée aux discussions entre utilisateurs.
- Page principale où les utilisateurs sont accueillis. Ils peuvent accéder à cette page en cliquant sur le bouton Home de la liste à gauche ou sur le logo du site. De plus, ils peuvent envoyer un nouveau message s'ils sont connectés au site au préalable et voir la liste des messages récemment postés. Cette page contient aussi un Header avec le logo du site, une zone pour rechercher des mots-clés dans des messages ou des messages envoyés entre 2 dates, et une zone d'authentification contenant des boutons de connexion ou d'inscription.
- Page Profil des utilisateurs affichant la photo de profil de l'utilisateur, le nom, le prénom et sa biographie. De plus, il y a plusieurs sous-onglets tels que Informations qui affiche des informations supplémentaires modifiables tels que le genre, l'âge, la date de naissance, l'email, l'adresse et la profession; Favoris qui liste les messages mis en favoris par l'utilisateur; Liste d'amis qui liste les amis de l'utilisateur et Liste des messages envoyés qui liste l'ensemble des messages envoyés par l'utilisateur.

Côté Serveur (dossier serveur):

- Gestion CRUD complète des projets et messages.
- API robuste permettant l'interaction fluide entre le client et le serveur.
- Stockage de tout type de données (les images également)
- Récupération des données depuis MongoDB.

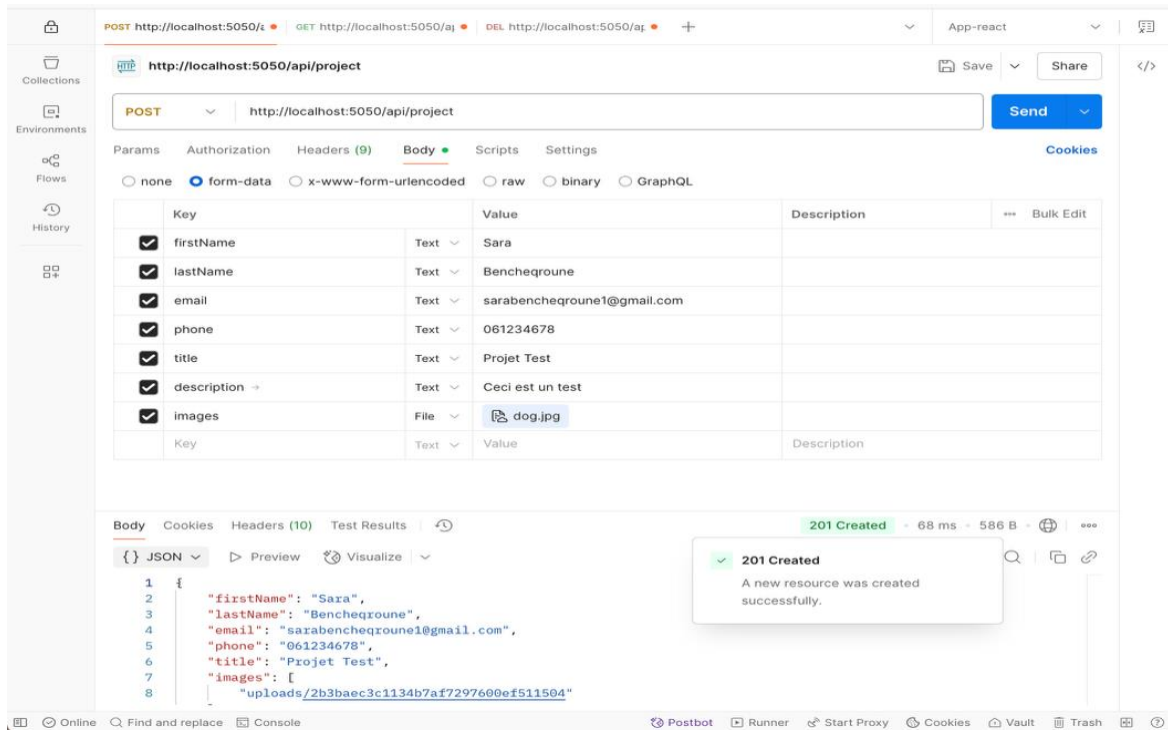
```
sarabenchegroune@Saras-MacBook-Pro-2 backend % node server.js

Serveur lancé sur http://localhost:5050
MongoDB Atlas connecté
```

4. Communication client-serveur :

Utilisation de `fetch` pour les appels API côté client, encapsulés dans des fonctions spécifiques comme `createProject`.

Exemple précis : la création de projets utilise la route `POST /api/project`, qui enregistre les données et images transmises par le client, et retourne un ID utilisé pour naviguer vers le chat lié au projet.

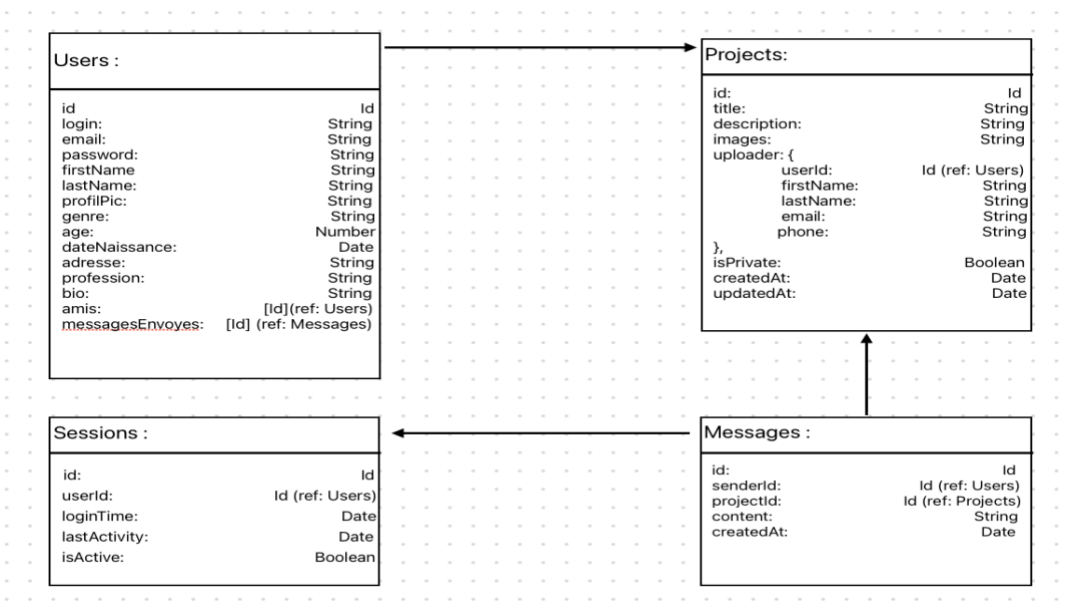


5. Schéma logique MongoDB :

Deux modèles principaux : Project et Message.

- o Project stocke des informations telles que titre, description, images, et informations utilisateur.
- o Message gère les interactions utilisateur-projet avec des champs pour l'identifiant utilisateur, projet et contenu des messages.

Relations clairement définies permettant une récupération efficace et organisée des données.



6. Répartition du travail :

Sara :

- Configuration complète de MongoDB Atlas.
- Développement des routes backend.
- Conception et développement de composants frontend (formulaires, authentification, prévisualisation, page principale....)
- Validation des API via tests Postman.
- connexion entre Frontend et le Backend

Jérémy:

- Construction et modélisation de l'architecture du site
- Développement de la partie frontend.
- Conception et développement de composants frontend (formulaires, authentification, page principale, profil)

7. Problèmes rencontrés :

Difficultés initiales avec la synchronisation entre les états React et les réponses serveur.
Problèmes de configuration MongoDB Atlas liés aux restrictions d'accès et à la gestion des images.
Défis d'intégration des requêtes Postman pour assurer un bon fonctionnement des routes.
Problème d'une bonne organisation de la structure de la page.
Problème de fusionnement entre les 2 parties

8. Améliorations possibles :

Implémentation d'une validation supplémentaire côté serveur pour renforcer la sécurité.
Gestion optimisée des erreurs côté client avec des notifications utilisateur plus précises.
Ajout d'option de navigation supplémentaire pour faciliter l'utilisation de l'app.
Ajout de fonction de changement de mot de passe si oublié.
Ajout de fonctions de connexion supplémentaires (ex : Google Gmail, etc...)

9. Choix de modélisation :

Modélisation légèrement différente du cours pour simplifier les interactions utilisateur-projet (suppression des couches intermédiaires non essentielles).
Choix d'une architecture simple, efficace, et facilement évolutive.